

Assignment 2

Submission. Your submission will be a folder on GitHub with the following structure:

```
assignment2/
  data/
    kennedy/...
    johnson/...
    unlabeled/...
  scripts/
    nb.py
  discussion.md
  conf.jpg
```

Problem 1. In this problem, we will write a Naive Bayes classifier to perform authorship identification over some speeches by President Lyndon B. Johnson and President John F. Kennedy. We will write these functions in an imperative programming style, but as we will see in future labs, using object-oriented programming would make for a cleaner design. As you work through this problem, you may find the following workflow useful: open up VS Code, and view both the Python file and a terminal in your single window. As you write code in the Python file, you can check your work in the terminal. It’s an effective practice to write small snippets of code and verify your work often. You may also want to use Python’s `input()` function, which halts execution at that line of code until you enter a newline on your keyboard. In the same vein, you can use VS Code to place breakpoints in your code, which have the same effect. To do this, you simply click to the right of the line number, placing a red dot to signify the breakpoint. **All discussion questions should be answered in `discussion.md`.** The `.md` ending stands for “markdown.” Markdown files are very similar to text-formatted files, ending in `.txt`, that you might be familiar with already.

- (A) Download the directory from Canvas called `assignment2`. You will notice there are three subfolders within the `data` folder. One folder is called `kennedy` and contains works by John F. Kennedy. Similarly, the folder `johnson` contains Johnson’s works. The goal in this problem will be to write a Naive Bayes classifier to determine the authorship of the unlabeled works in the `unlabeled` folder.
- (B) If you only had access to the prior probabilities, would you be more likely to guess that Kennedy or Johnson authored an unlabeled paper? Why?
- (C) Our first step is to organize the data within these directories in some sensible way. We’ll do this using a `pandas DataFrame`. You will implement the missing code in `build_dataframe`. Specifically, follow the comments labeled “TODO” to fill in the missing code. Once you’ve finished writing the function, take a look at the data and comment on what you find. You don’t have to perform an extensive exploratory data analysis, but you have a data structure that contains text from two “authors,” so you should take some time to look through the data. Some of the questions you may want to consider are: Do one author’s documents tend to be shorter in length? What are the most common words used by each author? Do either of the authors tend to start or end their works in a consistent way?
- (D) Now, you’ve structured your data and taken some time to get a feel for each author’s works. Our next step is to complete the code for the `train_nb` function. Here, we will write the training step for Naive Bayes, which will incorporate Lidstone smoothing. Although, in practice, it may be uncommon for you to write an algorithm from scratch, doing so has the tremendous benefit of you translating written ideas and pseudocode into a programming language. This skill must be developed over time, and we will start here. In `train_nb`, the core functionality—computation of the priors and likelihoods—is missing. Follow the TODOs for guidance.
- (E) What are your two prior probability estimates? What is the shape of the matrix storing your likelihoods? What happens when you vary the smoothing hyperparameter `alpha`?
- (F) What are the predicted authors for each of the unlabeled works?

Problem 2. In this problem, we’ll repeat Problem 1, but we’ll use the Naive Bayes classifier from the `scikit-learn` library.

- (A) Fill in the missing code to write the Naive Bayes classifier using `scikit-learn`. How do your predictions in Problem 1 compare with the `scikit-learn` implementation of Naive Bayes?

Problem 3. Now, we will evaluate your two models.

- (A) Report the accuracy and F1-score of both the Naive Bayes classifier you created and the one off-the-shelf from `scikit-learn` in your `discussion.md` file.
- (B) Print a confusion matrix for both the Naive Bayes classifier you created and the one off-the-shelf from `scikit-learn`. Save the plot as `conf.jpg` (or other image format). For each classifier, what do you notice from the confusion matrix?

Grading Rubric.

Problem	Maximum Score	Your Score	Comments
Problem 1	4		
Problem 2	1		
Problem 3	1		

Total	6		
-------	---	--	--

Grading Guide used by the TA Team.

- Problem 1: Students should have written code for a Naive Bayes classifier (4 points).
 - Does the overall Python module look good?
 - Did students create data structures to hold the text data?
 - Are the priors calculated correctly?
 - Are the likelihoods calculated correctly? Since this is the most challenging part of the assignment, this should carry much more weight.
 - Did students implement Lidstone smoothing correctly?
 - Did students discuss their results?
- Problem 2: Students should use the Naive Bayes classifier from the `scikit-learn` library (1 point).
 - Did students use the Naive Bayes classifier from the `scikit-learn` library as shown in the assignment 2 solution?
- Problem 3: Students should have written code to evaluate their approach versus the one from the `scikit-learn` library (1 point).
 - Did students report accuracy and F1-scores?
 - Did students create a confusion matrix?